Advanced Terraform on AWS Lab 5 Modularization, Contracts and Validations

Contents

Expected Duration 1
Teaching Points 1
Phase 1 – Baseline: Review the monolith 2
Phase 2 – Introduce a network module 4
Phase 3 - Refactor without destruction using moved blocks 7
Phase 4 – Introduce a compute module 9
Phase 5 – Contracts and Validation 10
Phase 6 – Module-Level Parameterisation 12
Phase 7 – Zero-Diff Refactor Across Environments 14
Phase 8 – Clean Up 15

Expected Duration

This lab should take 45-55 minutes to complete

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Teaching Points

At the end of Module 4, we had a hardened, well-structured monolithic Terraform configuration. It deployed networking resources including a VPC, subnets, and security rules. The code enforces naming, cleans inputs, validates CIDRs, and uses remote state files across dev\test\prod environments. Module 5 now extends this deployment to include EC2 instances, before evolving this monolith into two clean, reusable modules. The goal is to restructure - not redesign - while producing zero infrastructure changes.

The Problem We Are Solving

The Module 4 configuration is correct but not modular. All logic lives at the root. It cannot be reused or easily extended. Real-world Terraform uses modules for separation of concerns, flexibility, and reusability. We will now break the monolith into modules without changing deployed resources. When you move resources from the root into modules, their terraform addresses change. Terraform interprets this as deletion and recreation. This churn is expected behaviour. We can use `moved` code blocks to tell Terraform that resources have been moved, thus preventing churn.

Key principles:

- Root modules should behave like thin orchestrators.
- Module contracts are defined by inputs (variables), validation, and outputs.
- Validation belongs both at the root (external inputs) and module level (assumed invariants).
- Refactors should aim for zero-diff plans – behaviour must not change when moving to modules.

Lab Rules

- Work only inside the **lab5\guided** folder for the main configuration.
- Always specify **-var-file=<env>.tfvars** when deploying to dev\test\prod.
- Aim for a zero-diff plan after modularization – investigate any unexpected changes.

Prerequisites

This lab assumes the presence of a remote state backend, as created in Lab 4. If this does not exist in your lab AWS environment, then complete **Phase 2** of **Lab 4** before continuing.

Phase 1 – Baseline: Review the monolith

Lab 5 starts from the final Lab 4 configuration, but with one small extension.

The Lab 4 code deployed the shared AWS foundation: VPC, subnets, security group, and security group rules. For Lab 5, the starting configuration has been extended with a small EC2 workload so that the later modularisation has two natural boundaries:

- foundation infrastructure: VPC, subnets, security group, and rules
- compute workload: EC2 instances that depend on the foundation

The following changes have been made before the lab begins:

- A data source has been added to look up the latest Amazon Linux 2023 AMI in the configured AWS region.

- A new instances variable has been added. This is a map of named EC2 instances rather than a count-based value, so each instance has a stable Terraform key.
- One or more aws_instance resources are created using for_each over var.instances.
- Each EC2 instance is deployed into the **app** subnet and attached to the existing security group.
- The subnet_cidrs validation has been strengthened to require an app subnet, because the EC2 workload depends on it.
- The dev, test, and prod tfvars files now define different instance maps, allowing each environment to deploy a different number of EC2 instances without using ordinal count indexes.
- The instance map is deliberately normalized before use. Minor formatting issues such as accidental uppercase letters or surrounding spaces are managed automatically with locals. Values that affect behaviour, such as unsupported instance types or duplicate names after normalization, are rejected through validation.

This keeps the focus on Terraform refactoring rather than AWS compute. The EC2 instances exist only to give the later compute module something realistic to manage.

This phase establishes a behavioural baseline. The goal is not to change anything yet, but to capture what "correct" looks like before refactoring. Every later plan will be judged against this baseline.

1. In **lab5\guided**, update **providers.tf** with your remote storage S3 bucket **name**.
2. Run ...

   ```
   cd c:\qatfadvaws\lab5\guided
   terraform init -reconfigure -backend-config="key=dev.tfstate"
   terraform apply --var-file=dev.tfvars --auto-approve
   ```

   This is now your baseline deployment before modularization.

Verification

3. Confirm deployment of the twenty three expected resources by running

   ```
   terraform state list
   ```

```
PS C:\qatfadvaws\lab5\guided> terraform state list
data.aws_ami.amazon_linux
aws_instance.web["web01"]
aws_route_table.main
aws_route_table_association.subnet["app"]
aws_route_table_association.subnet["data"]
aws_route_table_association.subnet["mgmt"]
aws_security_group.main
aws_subnet.subnet["app"]
aws_subnet.subnet["data"]
aws_subnet.subnet["mgmt"]
aws_vpc.main
aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.0.50/32"]
aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.1.0/24"]
aws_vpc_security_group_egress_rule.rule["allow-https-partner|tcp|443|172.16.2.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.0.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.1.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.10.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-http-office|tcp|80|10.0.2.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|10.0.0.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|10.0.1.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|10.0.10.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|10.0.2.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|172.16.1.0/24"]
aws_vpc_security_group_ingress_rule.rule["allow-https-office|tcp|443|172.16.2.0/24"]
```

These are the same resources you created in Lab4 with the addition of a data block to identify instance AMIs and a single EC2 instance “web01”

Takeaway

You now have a working baseline plan against which you can compare the modularized configuration. Any behavioural change introduced by refactoring will be visible as a diff from this plan.

Refactoring Terraform should always start with a known-good plan. If you do not know what “no change” looks like, you cannot detect accidental change later.

Pause for thought

Before you begin, think about the following:

- Where are the natural boundaries for modules in your current configuration?
- Which resources belong to “network” concerns, and which belong to “compute” concerns?

Phase 2 – Introduce a network module

In this phase, we intentionally break the configuration to rebuild it correctly. Terraform will show errors as we progress through this phase. This is expected.

Moving resources into a module changes their Terraform addresses. Terraform will interpret this as “old resources removed, new resources added.” This is expected behaviour, not a mistake.

4. Create folder **lab5\guided\modules\network**
5. Copy **lab5\guided\main.tf** into **lab5\guided\modules\network**
6. In **lab5\guided\modules\network\main.tf**, delete resource blocks..

   data.aws_ami.amazon_linux
   resource.aws_instance.web

7. In **lab5\guided\main.tf**, delete **all** workflow blocks, **except**

   data.aws_ami.amazon_linux
   resource.aws_instance.web

8. Expect problems to be flagged in **modules\network\main.tf** due to references to undeclared variables.
9. Create file **modules\network\variables.tf** and populate with...

```
variable "prefix" {}
variable "base_tags" {}
variable "region" {}
variable "vpc_cidr" {}
variable "subnet_cidrs" {}
variable "allow_groups" {}
variable "security_group_rules" {}
```

   Note: These untyped variables will be strengthened later.

10. Copy **lab5\guided\locals.tf** into **lab5\guided\modules\network**
11. In **modules\network\locals.tf** remove root-only locals that will be passed into the module from root..

```
env_canon
project_canon
prefix
base_tags
```

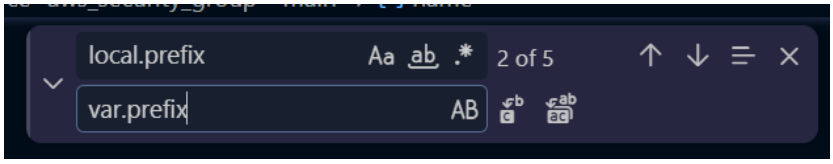
12. Still in **modules\network\locals.tf**, remove the unneeded compute local..

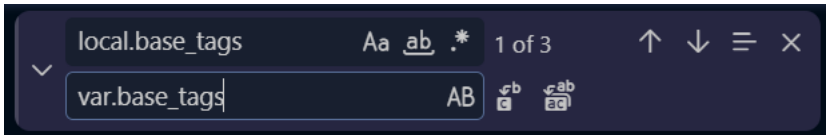
```
normalized_instances
```

13. In **modules\network\main.tf**, use find/replace (Ctrl+h) to replace ...

    `local.prefix` with `var.prefix` (5 replacements)

    `local.base_tags` with `var.base_tags` (3 replacements)





Pause for thought… Why are these changes necessary ?

14. Create file **modules\network\outputs.tf** and populate with

```
output "app_subnet_id" {
  value = aws_subnet.subnet["app"].id
}

output "security_group_id" {
  value = aws_security_group.main.id
}
```

This advertises module-created networking parameters back up to the root module

15. In **guided\main.tf**, replace the removed networking resources with a module block:

```
module "network" {
  source = "./modules/network"

  prefix              = local.prefix
  base_tags           = local.base_tags
  region              = var.region
  vpc_cidr            = var.vpc_cidr
  subnet_cidrs        = var.subnet_cidrs
  allow_groups        = var.allow_groups
  security_group_rules = var.security_group_rules
}
```

16. Still in **main.tf**, locate the **aws_instance.web** resource block, and update the **subnet_id** and **vpc_security_group_ids** parameters to reference the network module outputs …

```
subnet_id = module.network.app_subnet_id
vpc_security_group_ids = [module.network.security_group_id]
```

17. Re-initialize and plan…

```
cd c:\qatfadvaws\lab5\guided
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
```

Verification

Confirm that the plan proposes twenty three **replacements**.

Although no infrastructure has actually changed, Terraform tracks resources using their **state address**, not their physical existence in AWS. Before modularisation, resources were stored in the state file using addresses such as:

aws_vpc.main
aws_subnet.subnet["app"]
aws_security_group.main

After moving these resources into module.network, their addresses become:

module.network.aws_vpc.main
module.network.aws_subnet.subnet["app"]
module.network.aws_security_group.main

Terraform sees the original addresses disappear and new addresses appear. Without further guidance, it assumes the old resources should be destroyed and the new resources created. This results in a plan proposing 23 destroy/create operations, even though the underlying AWS resources have not changed.

You may notice that the AMI data source and EC2 instance remain in the root module and have **not** changed address. However, the EC2 instance depends on resources that Terraform currently believes are being replaced, including the subnet and security group. As a result, Terraform also proposes changes to the EC2 instance because its dependencies appear to be changing

Takeaway

You have now extracted networking concerns into modules\network but have introduced potential configuration and state churn. Modules do not inherit context. Every value a module needs must be passed explicitly as an input, even if that value previously existed in the root module.

Phase 3 - Refactor without destruction using moved blocks

This phase teaches you how to use Terraform's **moved** block feature to avoid destroying real infrastructure when refactoring resources into modules. Terraform tracks resources by address, so modularization changes addresses and may trigger destructive behaviour unless a moved block is provided.

A moved block performs its migration exactly once. After Terraform has updated the state for all environments, the block becomes a no-op. However, in real projects you should keep

the moved block in your Terraform code until you are certain that all environments—and all team members—have applied the updated configuration.

If a moved block is removed too early, Terraform will assume the old address was dropped and may try to destroy and recreate existing infrastructure.

In production, moved blocks typically stay in the codebase for weeks or months and are removed only during a deliberate, controlled cleanup refactor once all state files have been fully migrated.

The moved.tf file supplied with this lab was generated by examining the existing Terraform state.

Running **terraform state list** displays the resource addresses currently stored in the state file. During modularisation, these addresses change because resources move from the root module into module.network.

For example: **aws_vpc.main** becomes **module.network.aws_vpc.main**

The provided **moved.tf** file maps each original state address to its new module address. This allows Terraform to update the state without destroying and recreating the underlying AWS resources.

The AMI data source and EC2 instances are not included because they have not yet been moved into a module.

Implementation

18. A **moved.tf** file has been provided, detailing the resource moves, but it is currently inactive. Comment out lines **1** and **111** to activate the move of resources into the network module. (Leave the remainder of the file commented-out)

    Real-world note
    The `moved.tf` file supplied with this lab has been prepared in advance. In production environments, teams rarely type large numbers of `moved` blocks by hand. Instead, they commonly generate candidate mappings from `terraform state list` using a script and then review them before committing. Terraform cannot generate these mappings automatically because only the engineer knows which resources have genuinely been moved and which have been renamed or replaced.

19. Run `terraform plan --var-file=dev.tfvars`

Verification

This should result in a zero-difference plan.

Takeaway

Using 'moved' allows modularization without churn. Any time you refactor Terraform structure without intending to change infrastructure, expect to pair that refactor with explicit state migration instructions.

Phase 4 – Introduce a compute module

Try in yourself (Optional)

20. Based on the processes outlined in phases 2 and 3 above, move the compute resources currently in the root module into a dedicated '**compute'** module with zero-difference, uncommenting lines 113 and 118 of **moved.tf** when appropriate.

Step-by-step

This phase deliberately repeats the previous pattern with different resources. The goal is not novelty, but reinforcement: modularisation follows the same principles regardless of resource type.

21. Create folder **lab5\guided\modules\compute**
22. Copy **lab5\guided\main.tf** into **lab5\guided\modules\compute**
23. In **compute\main.tf**, delete the **module.network** block.
24. In **guided\main.tf**, delete the data and instances blocks..

    ```
    data.aws_ami.amazon_linux
    resource.aws_instance.web
    ```

25. Expect problems to be flagged in **compute\main.tf** due to references to undeclared variables.
26. Create file **compute\variables.tf** and populate with

    ```
    variable "prefix" {}
    variable "base_tags" {}
    variable "subnet_id" {}
    variable "security_group_id" {}
    variable "instances" {}
    ```

27. Copy **guided\locals.tf** into **modules\compute**
28. In **compute\locals.tf** remove **all** entries **except..**

    ```
    normalized_instances
    ```

29. In **compute\main.tf**, replace ...

    `local.prefix` with `var.prefix`
    `local.base_tags` with `var.base_tags`

30. Still in **main.tf**, locate the **aws_instance.web** resource block, and change the **subnet_id** and **vpc_security_group_ids** parameters to variables, the values of which will be passed down from root once it has derived them from the network module outputs …

```
subnet_id = var.subnet_id
vpc_security_group_ids = [var.security_group_id]
```

31. In the **guided\main.tf,** replace the removed compute resources with a module block:

```
module "compute" {
  source = "./modules/compute"

  instances         = var.instances
  subnet_id         = module.network.app_subnet_id
  security_group_id = module.network.security_group_id
  prefix            = local.prefix
  base_tags         = local.base_tags
}
```

32. Plan the changes…

```
cd c:\qatfadvaws\lab5\guided
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
```

    One replacement should be proposed.

33. Comment out lines **113** and **118** of the provided **moved.tf**
34. Run; **terraform plan --var-file=dev.tfvars**
35. This should show a zero-diff move to a modularization of compute

Phase 5 – Contracts and Validation

What changes here? Until now, validation lived mostly at the root. From this point onward, modules become responsible for defending their own assumptions. This is the transition from “working code” to “robust code.”

36. Open and review **modules\network\variables.tf** and **modules\compute\variables.tf**.
37. The variables needed for the modules have been defined, but there are untyped and have no validation checks being performed. At present the checks are being performed at the root module level. Whilst this works in principle, in practice the modules should ‘stand-alone’ with the root module performing minimal high-level verification of data that will be passed into the module.

Implementation

38. From **guided\variables.tf**, *copy* the following variable definition blocks into **network\variables.tf**;

    ```
    allowed_groups
    security_group_rules
    subnet_cidrs
    vpc_cidr
    ```

39. In **network\variables.tf**, remove the initially defined untyped variables matching those now fully defined in the previous step.

40. In **guided\variables.tf**; remove the validation portions of the variable definitions;

    ```
    allowed_groups
    security_group_rules
    subnet_cidrs
    vpc_cidr
    ```

41. From **guided\variables.tf**, copy the following variable definition block into **compute\variables.tf**;

    ```
    instances
    ```

42. In **compute\variables.tf**, remove the initially defined {} variables matching those now fully defined in the previous step.

43. In **guided\variables.tf**; remove the validation portions of the variable definitions;

    ```
    instances
    ```

44. In **guided\locals.tf**; remove **all** entries, **except**...

    ```
    env_canon
    project_canon
    prefix
    base_tags
    ```

45. Confirm these changes do not introduce churn;

    ```
    terraform plan --var-file=dev.tfvars
    ```

    No changes should be proposed

46. Introduce errors at the root module to verify that they are captured and we fail fast and early. Do this by altering values such as the allowed groups and subnet masks in **guided\dev.tfvars** and find the impact by running;

    ```
    terraform plan --var-file=dev.tfvars
    ```

47. Roll-back any changes made

Takeaway

You have strengthened the contracts for your modules. Validation now protects against misconfiguration at the module boundary, complementing root-level validation on raw inputs.

Phase 6 – Module-Level Parameterisation

The network and compute modules currently receive values passed down from the root module. These values describe the infrastructure that the root module is orchestrating, such as network ranges, subnet definitions and compute settings.

Alongside these root-level parameters, a child module may also need values that relate specifically to the module itself. In this phase, you will add a simple release marker to each module.

The release marker will:

- identify the module;
- record the module version;
- provide a visible change when a new module release is created;
- return the module version to the root module through an output.

The marker will use the provider-independent **terraform_data resource**, so no additional AWS resources will be created.

Add a release marker to the network module

48. Open **network\main.tf** and add the following resource:

```
resource "terraform_data" "module_version" {
  input = {
    module  = "network"
    version = "Unversioned"
    change  = "Initial network module release"
  }
}
```

    This resource does not create anything in AWS. It stores simple release information in Terraform state.

    The values are specific to the network module and are therefore defined inside the module rather than passed down from the root module.

Output the network module version

49. Open **network\outputs.tf** and add the following output:

```
output "module_version" {
  description = "Version of the network module"

  value = terraform_data.module_version.output.version
}
```

The terraform_data resource exposes its input value through the output attribute. The output therefore returns 'v1.0.0' to the calling root module

Add a release marker to the compute module

50. Open **compute\main.tf** and add the following resource:

```
resource "terraform_data" "module_version" {
  input = {
    module  = "compute"
    version = "Unversioned"
    change  = "Initial compute module release"
  }
}
```

As with the network module, this resource exists only as a Terraform-managed release marker. It does not create or modify any AWS infrastructure.

Output the compute module version

51. Create file **compute\outputs.tf** and add the following output:

```
output "module_version" {
  description = "Version of the compute module"

  value = terraform_data.module_version.output.version
}
```

This makes the compute module version available to the root module as: module.compute.module_version

Expose the module versions from the root module

52. Create file **guided\outputs.tf** file and add the following:

```
output "deployment_summary" {
  description = "Summary of the deployed environment and module versions"

  value = {
    environment = var.environment

    modules = {
```

```
      network = module.network.module_version
      compute = module.compute.module_version
    }
  }
}
```

The root module now collects and outputs to screen, the version information returned by both child modules.

Format and validate the configuration

53. From **lab5\guided**, run:

```
terraform fmt -recursive
terraform validate
```

Confirm that the configuration is valid.

Apply the changes

54. Run: `terraform apply --var-file=dev.tfvars --auto-approve`

After the apply completes, adding the two new resources, Terraform should display outputs similar to:

network_module_version = "Unversioned"
compute_module_version = "Unversioned"

You can display the outputs again by running `terraform output`

Phase 7 – Zero-Diff Refactor Across Environments

55. Plan and Apply workflows for all three environments, using the same backend keys as previously …

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform plan --var-file=dev.tfvars
terraform apply --var-file=dev.tfvars --auto-approve
```

```
terraform init -reconfigure -backend-config="key=test.tfstate"
terraform plan --var-file=test.tfvars
terraform apply --var-file=test.tfvars --auto-approve
```

```
terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform plan --var-file=prod.tfvars
terraform apply --var-file=prod.tfvars --auto-approve
```

56. The dev environment deployment should be a no-diff as it is already deployed, and no module-level tagging was performed. The test and prod environments are fresh deployments and should succeed. The moved.tf code should be a no-op.
57. Use the portal to confirm all three environments deployments now exist simultaneously, tracked by a unique backend statefile in a shared storage account.
58. Given that, in this lab, there are no other team members who may be relying on the 'moved' blocks, you can safely delete **moved.tf** as it is now a pure no-ops across all environments.

You have confirmed that the modularization is behaviour-preserving across dev, test, and prod. Zero-diff refactors provide confidence that structural improvements have not changed the actual infrastructure.

Stretch Challenge

A colleague suggests that the network module should expose every resource ID it creates so that future modules have maximum flexibility.

Do you agree?

If not, identify two or three outputs that you believe should be exposed and justify your choices.

Phase 8 – Clean Up

59. Ensure you are in **lab5\guided** and run;

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform destroy --var-file=dev.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=test.tfstate"
terraform destroy --var-file=test.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform destroy --var-file=prod.tfvars --auto-approve
```

Verification

60. Use the Azure portal to confirm that all lab resources have been destroyed except for the remote backend.

Takeaway

You have completed the modularization lab. You now have a repeatable pattern for turning a flat configuration into a set of reusable, validated modules while preserving behaviour across environments. We will next look at invoking this modular approach as the starting

pattern rather than by a refactoring process as was performed here. We will also discuss module version and remote hosting of modules.

Copyright